

OOP Project

Hook Master

Mattia Evangelisti
Fabio Malucelli
Matteo Raganini
Elia Soldati

1.1 Descrizione e requisiti.....	3
1.2 Modello del Dominio.....	4
2.1 Architettura.....	6
2.2 Design dettagliato.....	7
Mattia Evangelisti.....	7
Comportamento di movimento intercambiabile dei pesci.....	7
Creazione dei pesci e filtraggio in base alle condizioni meteo.....	8
Fabio Malucelli.....	9
Gestione semplificata degli stati dell'amo.....	10
Comunicazione degli eventi di dominio.....	11
Difficoltà del minigioco dipendente dal meteo.....	12
Creazione centralizzata del minigioco.....	13
Matteo Raganini.....	14
Gestione dei potenziamenti.....	14
Struttura dei potenziamenti.....	15
Creazione dei potenziamenti.....	16
Elia Soldati.....	18
Sistema delle collisioni.....	18
Controller phase system.....	19
Sistema di salvataggio.....	20
Pesci predatori.....	21
3.1 Testing automatizzato.....	22
3.2 Note di sviluppo.....	23
Mattia Evangelisti.....	23
Fabio Malucelli.....	23
Matteo Raganini.....	24
Elia Soldati.....	24
4.1 Autovalutazione e lavori futuri.....	25
Mattia Evangelisti.....	25
Fabio Malucelli.....	25
Matteo Raganini.....	26
Elia Soldati.....	26
4.2 Commenti per i docenti.....	27
Elia Soldati.....	27
Appendice A - Guida Utente.....	28

Analisi

1.1 Descrizione e requisiti

Il progetto mira allo sviluppo di un gioco arcade 2D chiamato "Hook Master" dove il giocatore si cimenta nella pesca. Per gioco arcade si intende un gioco caratterizzato da un gameplay rapido e focalizzato sull'azione. Il giocatore controlla un amo che si muove liberamente all'interno di un'area di gioco con lo scopo di catturare diverse specie di pesci, ciascuna caratterizzata da: comportamenti, valori economici e pesi differenti. In caso di contatto con un pesce il giocatore deve completare un minigioco per effettuare la cattura. Catturando i pesci il giocatore riceverà una valuta utilizzabile nel negozio di gioco per comprare potenziamenti che lo aiuteranno a progredire.

Il gioco introduce inoltre un sistema di variazioni climatiche, che influenzano sia la difficoltà della cattura sia il comportamento di determinate specie di pesci e un ecosistema di prede e predatori.

Requisiti funzionali:

- Il giocatore deve poter muovere l'amo liberamente in tutte le direzioni all'interno dell'area di gioco, tramite input da tastiera.
- Quando l'amo entra in contatto con un pesce deve avviarsi un minigioco di cattura basato sul tempismo (Quick Time Event): il giocatore deve premere un tasto nel momento in cui l'indicatore si trova all'interno di una zona ristretta di successo.
- Ogni specie di pesce deve possedere attributi distinti (valore economico, difficoltà di cattura, peso, velocità, comportamento di movimento) che influenzano sia il minigioco di cattura sia la possibilità della cattura stessa.
- Alcuni pesci non devono poter essere catturati se il loro peso supera la capacità massima corrente dell'amo, capacità che può essere aumentata dallo shop.
- Il sistema deve prevedere un ecosistema di predatori che interagiscono con le specie di pesci più piccole.
- Il gioco deve includere condizioni climatiche variabili nel tempo, che influenzano la velocità dei pesci, la difficoltà di cattura e la possibilità di spawn di determinate specie.
- Il giocatore deve poter accedere ad un negozio in cui spendere le monete guadagnate per acquistare potenziamenti che migliorano le prestazioni dell'amo e semplificano la difficoltà del minigioco.

Requisiti non funzionali:

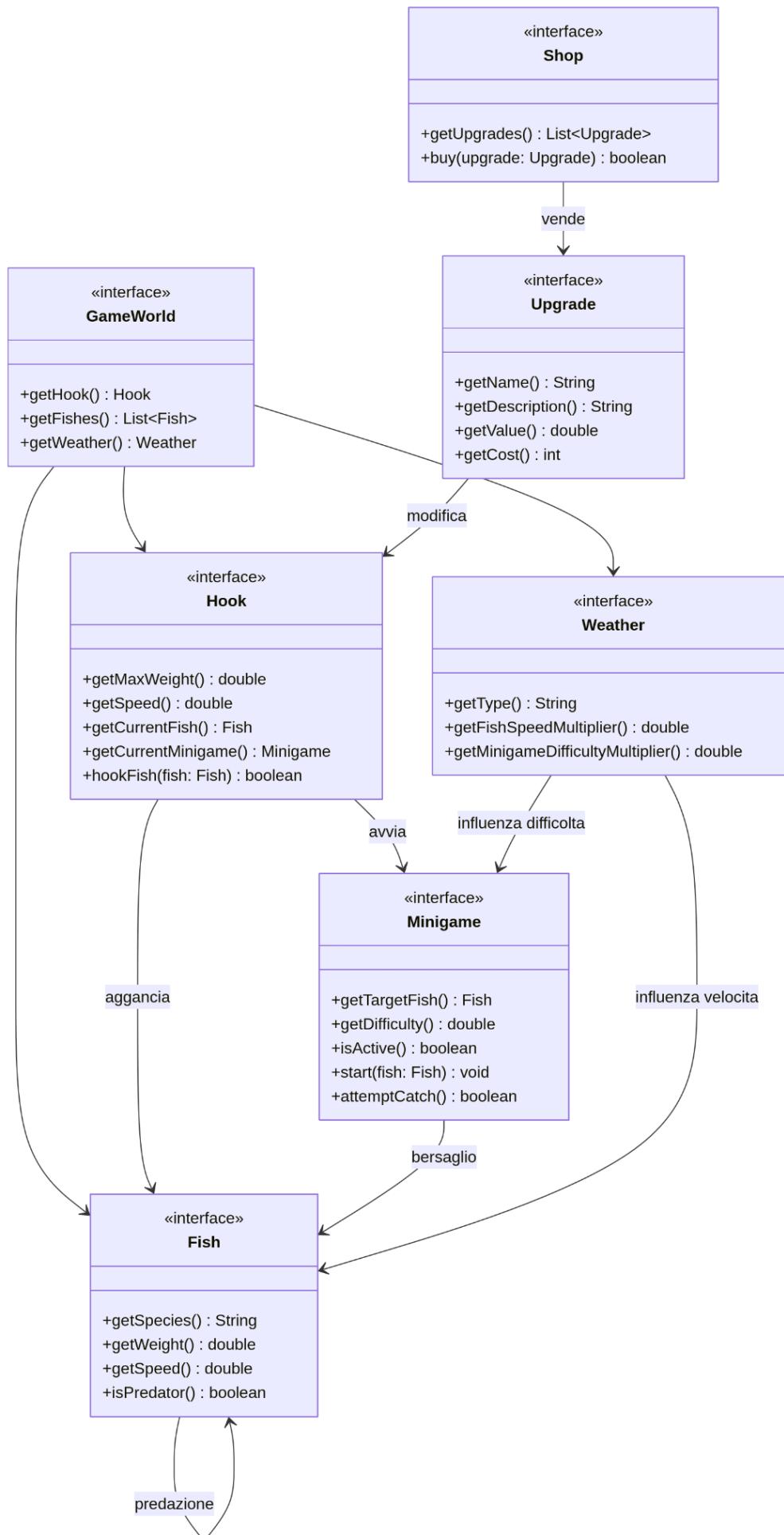
- Bisogna garantire un gameplay fluido e reattivo, con tempi di risposta minimi ai comandi.
- Il gioco deve gestire le collisioni e interazioni tra i numerosi elementi di gioco in modo coerente.
- Ci deve essere un'organizzazione che permetta di ampliare il gioco con facilità (nuove specie di pesci e nuovi potenziamenti).
- Portabilità su Linux, Windows, macOS.

1.2 Modello del Dominio

In questa sezione si descrivono le entità che compongono il dominio applicativo di Hook Master, indipendentemente da come esse vengano realizzate.

Entità principali:

- **Hook:** rappresenta l'amo con cui il giocatore pesca i pesci, contiene i valori per il peso massimo e velocità, si occupa di avviare il minigioco.
- **Fish:** rappresenta un pesce (predatore o meno) con una sua specie e i suoi valori specifici (peso e velocità). Possono anche mangiarsi a vicenda nel caso dei predatori.
- **Minigame:** viene avviato quando il giocatore aggancia un pesce e va completato per catturarlo e guadagnare il suo valore economico. È un minigioco basato sul tempismo.
- **Shop:** permette al giocatore di comprare i potenziamenti.
- **Upgrade:** i potenziamenti che permettono al giocatore di migliorarsi spendendo le monete guadagnate. Vanno a modificare i valori specifici dell'amo.
- **Weather:** il tempo atmosferico, si presenta casualmente durante il gioco, e influenza la velocità dei pesci e la difficoltà del minigioco.
- **World:** rappresenta il mondo di gioco in cui i pesci nuotano e si alternano i vari stati del tempo atmosferico.



Design

2.1 Architettura

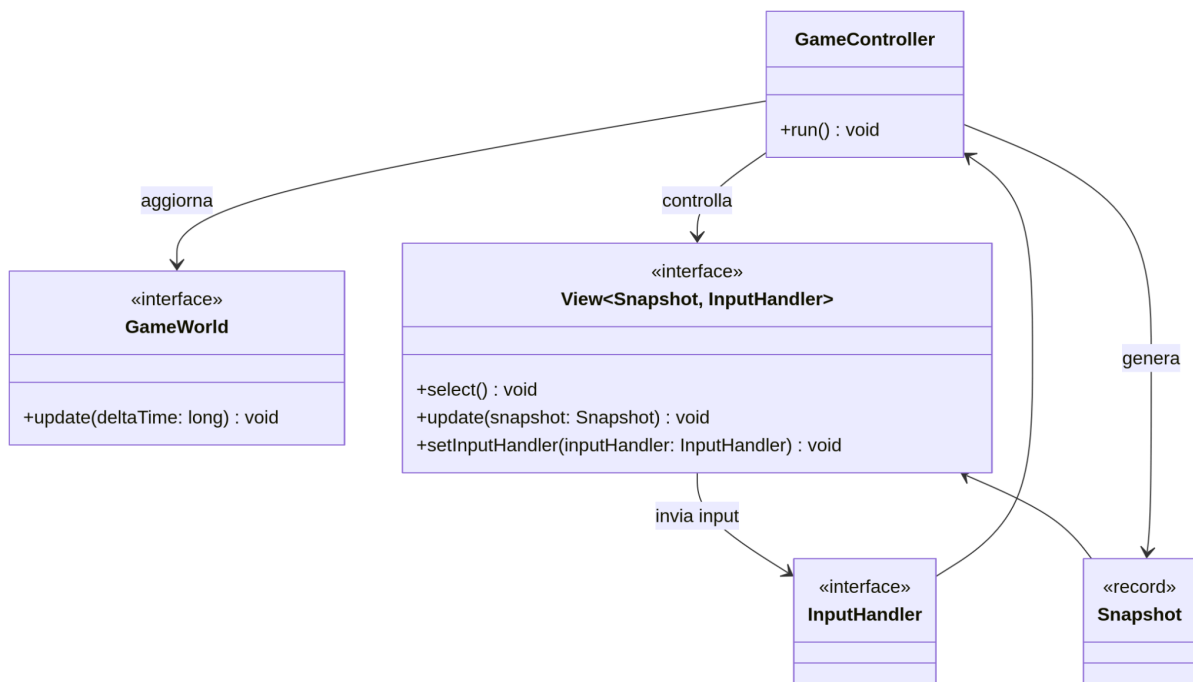
Abbiamo scelto di usare il pattern architetturale MVC per favorire una separazione tra logica, interfaccia e controllo.

Il **Model** contiene la logica del gioco e si occupa di mantenerne e aggiornarne lo stato. L'interfaccia principale che funge come punto d'ingresso al model è il *GameWorld*, che è responsabile dell'avanzamento del tempo nel mondo di gioco.

La **View** è la componente responsabile della visualizzazione dei dati contenuti nel model. Il suo punto d'ingresso principale nella nostra architettura è l'interfaccia *View<Snapshot, InputHandler>* che rappresenta una view responsabile di una determinata fase del gioco (per esempio menù, oppure lo shop, o ancora il mondo di gioco). *Snapshot* rappresenta il tipo di dato che la view consuma per visualizzare l'interfaccia (per esempio nel caso dello shop ci saranno contenuti gli upgrade). *InputHandler* rappresenta l'interfaccia che la specifica view richiede per passare gli input al controller.

Si tiene a specificare che *Snapshot* e *InputHandler* sono due categorie di interfacce e non due reali interfacce esistenti nel codice: ogni view possiede una sua specifica interfaccia per ognuna delle due categorie che sono poi legate ad essa attraverso i due parametri generici dell'interfaccia *View*. Questo è dovuto al fatto che ogni fase del gioco, e quindi ogni view, ha un suo specifico set di dati necessari alla visualizzazione e un suo specifico set di input che vengono utilizzati.

Il **Controller** infine, è il componente che funge da collegamento tra model e view: è responsabile di tradurre i dati esposti dal model per la view in un formato ad essa comprensibile, e per l'altro verso è responsabile di tradurre gli input ricevuti dalla view (attraverso l'*InputHandler*) in comandi comprensibili al model. È anche responsabile di far avanzare lo stato del model



[Link](#)

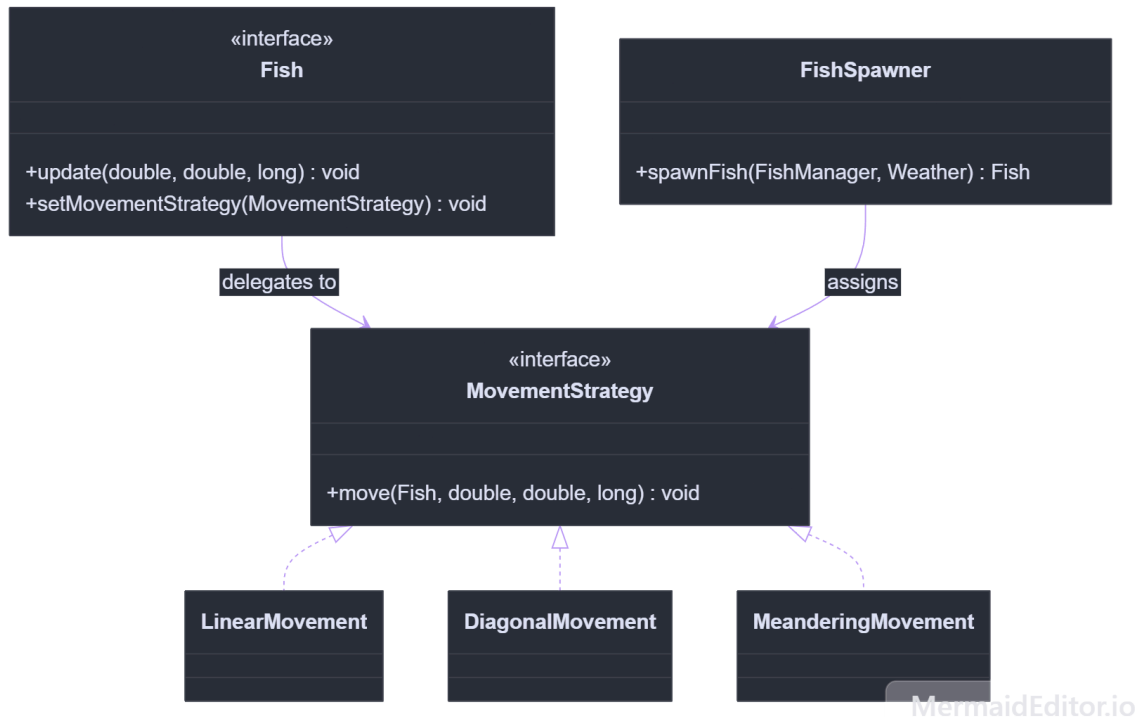
2.2 Design dettagliato

Mattia Evangelisti

Comportamento di movimento intercambiabile dei pesci

Problema: I pesci devono poter esibire pattern di movimento differenti e intercambiabili senza che l'aggiunta di un nuovo tipo di movimento richieda modifiche alla classe che rappresenta il pesce, né che quest'ultima debba conoscere i dettagli implementativi di ciascun algoritmo di movimento.

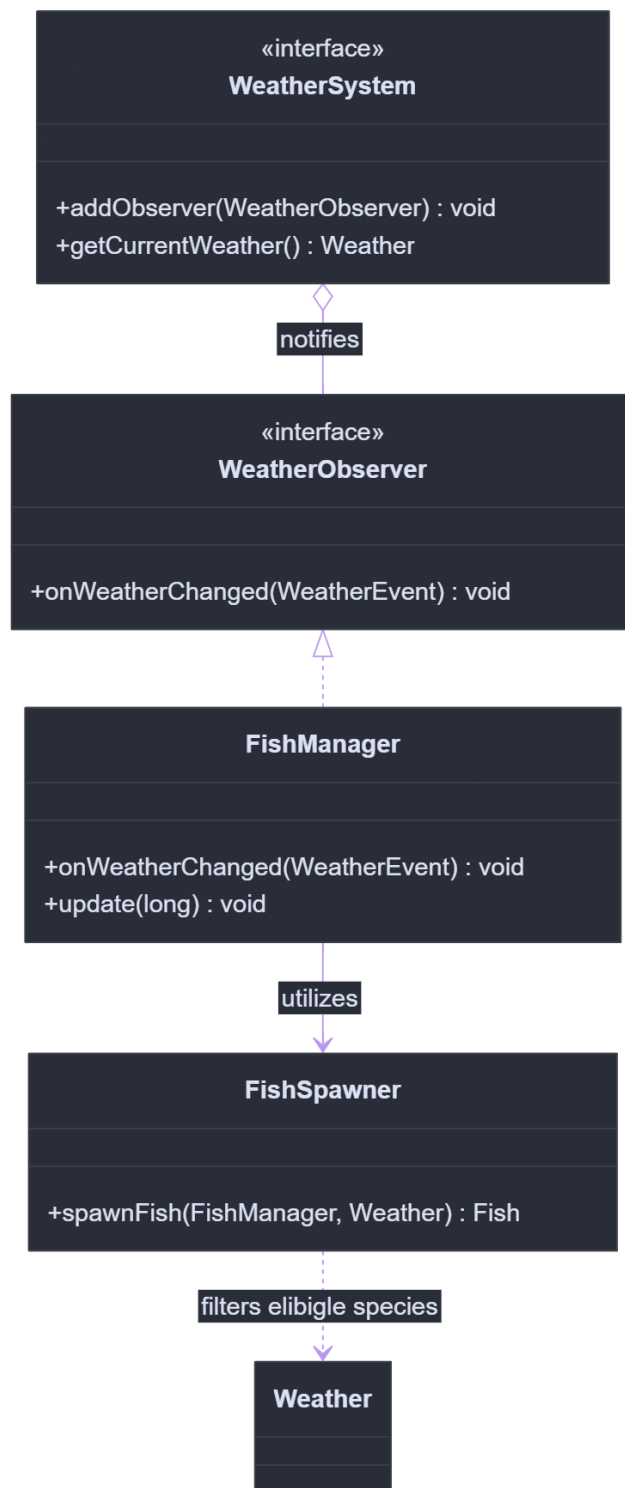
Soluzione: È stato applicato lo strategy pattern. L'interfaccia funzionale **MovementStrategy** definisce il contratto comune `move(...)`, mentre **Linear**, **Diagonal** e **MeanderingMovement** ne sono implementazioni concrete intercambiabili. Ogni istanza di **Fish** possiede una **MovementStrategy** corrente e le delega interamente la logica di aggiornamento della posizione ad ogni tick di simulazione, tramite il metodo `update(...)`. La scelta della strategia di movimento per un nuovo pesce avviene in **FishSpawner**, che ne sceglie una casualmente tra quelle disponibili al momento dello spawn; questo disaccoppia la logica di creazione del pesce dalla logica di movimento, permettendo di aggiungere nuove strategie senza toccare né Fish né le strategie esistenti.



Creazione dei pesci e filtraggio in base alle condizioni meteo

Problema: Alcune specie di pesci devono poter comparire solo durante determinate condizioni meteorologiche e tutti i pesci devono reagire dinamicamente ai cambiamenti meteo senza che il modulo di gestione dei pesci debba conoscere i dettagli implementativi del sistema meteo, né viceversa.

Soluzione: È stato applicato un observer pattern dove **FishManager** implementa l'interfaccia **WeatherObserver** e si registra come osservatore presso il **WeatherSystem** al momento della propria costruzione. Quando il sistema meteo notifica un cambiamento del tempo, **Fishmanager** aggiorna il moltiplicatore di velocità di tutti i pesci vivi e quelli che dovranno nascere. La stessa informazione sul meteo corrente viene passata come parametro a **FishSpawner** che filtra l'insieme delle specie eleggibili allo spawn in base al flag **StormOnly** di ciascun **FishType**.



MermaidEditor.io

Fabio Malucelli

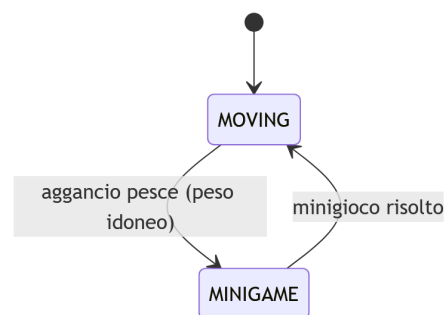
In questa sezione vengono analizzate le scelte progettuali relative al movimento dell'amo, alla gestione delle condizioni climatiche e al minigioco di cattura basato su tempismo(Quick Time Event).

L'obiettivo principale é stato garantire che il model (l'amo e le sue regole) restasse completamente disaccoppiato dal controller e dalla view, delegando a un'unica entità (l'amo stesso) la responsabilità di decidere autonomamente come reagire agli eventi che lo riguardano (collisioni, potenziamenti acquistati, cambiamenti climatici), così da mantenere il controller ridotto a pura delega degli input dell'utente.

Gestione semplificata degli stati dell'amo

Problema: Il comportamento dell'amo cambia significativamente a seconda della fase in cui si trova: quando si muove liberamente può accettare input di movimento e agganciare un pesce in caso di collisione, mentre durante lo svolgimento del minigioco di cattura deve restare fermo e ignorare qualunque tentativo di aggancio di un secondo pesce. Gestire questa distinzione tramite variabili booleane sparse avrebbe portato a condizioni duplicate e al rischio di stati incoerenti.

Soluzione: Ho modellato una enum HookState con due soli valori, MOVING e MINIGAME, usata all'interno dei metodi che gestiscono il comportamento dell'amo (avanzamento della posizione, gestione delle collisioni, minigioco). Si tratta di un'applicazione semplificata del pattern State: a differenza della forma classica del pattern, in cui ciascuno stato è una classe concreta che implementa un'interfaccia comune e incapsula il proprio comportamento in modo polimorfico, qui la selezione del comportamento avviene tramite uno switch unico sulla enum all'interno della classe HookImpl. Questa scelta l'ho ritenuta praticabile per via del numero ridotto di stati e di complessità, ma non elimina la logica condizionale come farebbe l'applicazione completa del pattern: qualora in futuro il numero di stati o la complessità del comportamento per stato dovesse crescere, sarebbe opportuno creare classi di stato polimorfiche separate.

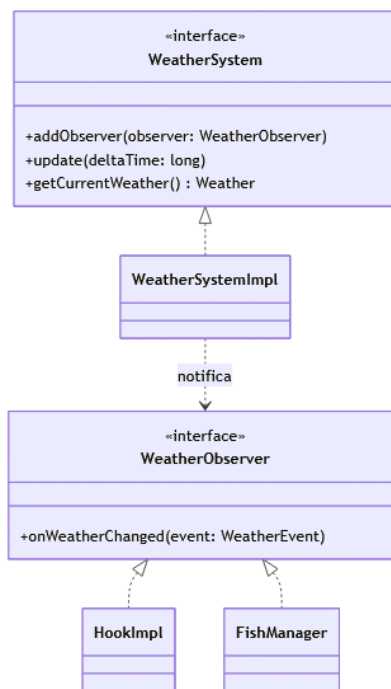


[UML-Property](#)

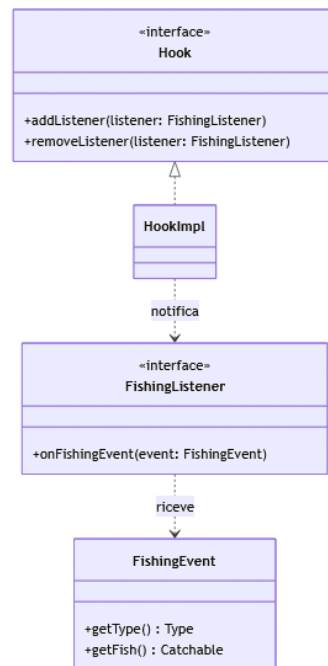
Comunicazione degli eventi di dominio

Problema: Diversi sottosistemi del gioco generano eventi rilevanti per altre parti del sistema, che non devono però conoscere i dettagli implementativi. Il sistema meteo deve poter informare qualunque entità interessata (l'amo, la gestione dei pesci, la view) di un cambiamento climatico; l'amo deve poter informare la view o futuri sistemi di ricompensa o statistiche di eventi come l'aggancio o la cattura di un pesce (senza che né il meteo né l'amo debbano conoscere direttamente chi li osserva).

Soluzione: Ho adottato il pattern Observer in due punti distinti del sistema. Nel primo caso, l'interfaccia WeatherSystem espone i metodi addObserver e removeObserver notifica gli interessati tramite l'interfaccia WeatherObserver ogni volta che il meteo cambia; sia l'amo sia il sottosistema di gestione dei pesci implementano un WeatherObserver, decidendo autonomamente come reagire al cambiamento (per l'amo questo si traduce in una maggiore difficoltà del minigioco di cattura). Nel secondo caso, è lo stesso amo a fungere da soggetto osservabile: espone addListener e removeListener per l'interfaccia FishingListener, e notifica un FishingEvent (tipizzato tramite enum: pesce agganciato, catturato, sfuggito) ogni volta che compie una transizione di stato rilevante. In entrambi i casi il chi viene osservato non conosce mai il tipo concreto dei propri osservatori.



[UML-Property](#)

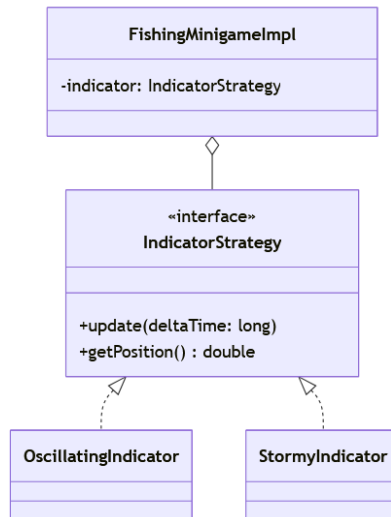


[UML-Property](#)

Difficoltà del minigioco dipendente dal meteo

Problema: Il minigioco di cattura consiste in un indicatore che si muove lungo una barra, il giocatore deve premere un tasto quando l'indicatore si trova all'interno di una zona bersaglio. Per la presenza di tempesta data dalle condizioni climatiche il movimento dell'indicatore è meno prevedibile e quindi più difficile da seguire, senza però duplicare la logica del minigioco per ogni possibile condizione climatica.

Soluzione: Ho applicato il pattern Strategy: il comportamento di movimento dell'indicatore è stato estratto da un'interfaccia dedicata, IndicatorStrategy, che espone un solo metodo per avanzare la posizione dell'indicatore nel tempo. Ho realizzato due implementazioni concrete: una a velocità costante usata in condizioni climatiche normali e una a velocità variabile nel tempo usata durante le tempeste, che rende il movimento dell'indicatore più imprevedibile. Il resto della logica del minigioco (individuazione della zona bersaglio, verifica se il giocatore ha premuto il tasto nel momento giusto) resta identica indipendentemente dalla strategia in uso.

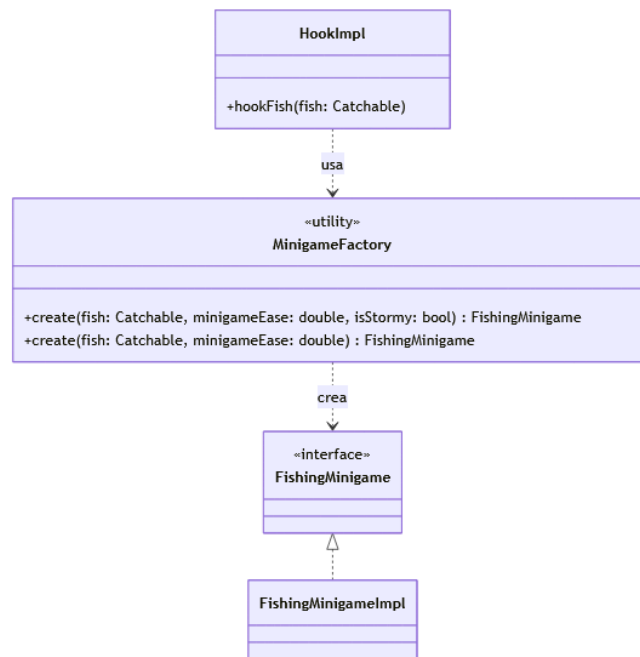


[UML-Property](#)

Creazione centralizzata del minigioco

Problema: Ogni volta che l'amo aggancia un pesce va creata un'istanza del minigioco di cattura scegliendo la strategia dell'indicatore corretta in base alla condizione meteo corrente. Se questa scelta fosse ripetuta in tutte le parti dove un pesce possa essere agganciato, si introdurrebbe duplicazione e un accoppiamento diretto tra l'amo e le implementazioni concrete delle strategie.

Soluzione: Ho applicato il pattern Factory Method tramite una classe di utilità dedicata, che espone un unico metodo statico di creazione: dati il pesce da catturare, un valore di difficoltà base e la condizione climatica, restituisce un'istanza pronta del minigioco, con la strategia dell'indicatore già selezionata internamente. L'amo non istanzia mai direttamente le implementazioni concrete delle strategie, né conosce come viene costruito il minigioco, si limita solo a invocare il Factory Method.



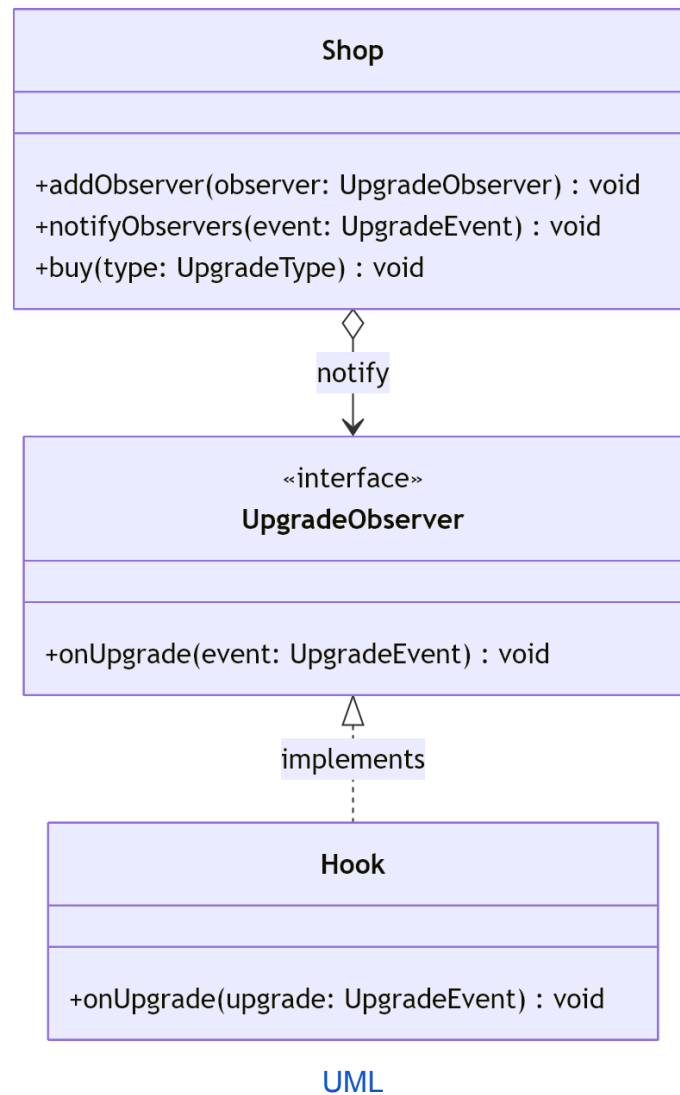
[UML-Property](#)

Matteo Raganini

Gestione dei potenziamenti

Problema: Il gioco prevede uno shop dove il giocatore può acquistare dei potenziamenti. Tali potenziamenti vanno a migliorare aspetti diversi del gioco (ad esempio il peso massimo che si può pescare o la difficoltà del minigioco). È necessario che lo shop comunichi l'avvenuto acquisto a tutti gli interessati senza creare un forte accoppiamento tra la logica dello shop e gli altri componenti del gioco.

Soluzione: Un'alternativa sarebbe stata mantenere in **Shop** riferimenti diretti a ogni componente da aggiornare, chiamando direttamente il metodo dopo un acquisto. Questa soluzione è stata scartata perché avrebbe accoppiato fortemente **Shop** e le componenti dipendenti dai potenziamenti, richiedendo di modificare **Shop** ogni volta che un nuovo potenziamento viene aggiunto. Ho quindi deciso di implementare il pattern **Observer**, per consentire una comunicazione uno a molti: **Shop** non conosce i singoli componenti da aggiornare, ma si limita a notificare una lista di osservatori registrati.



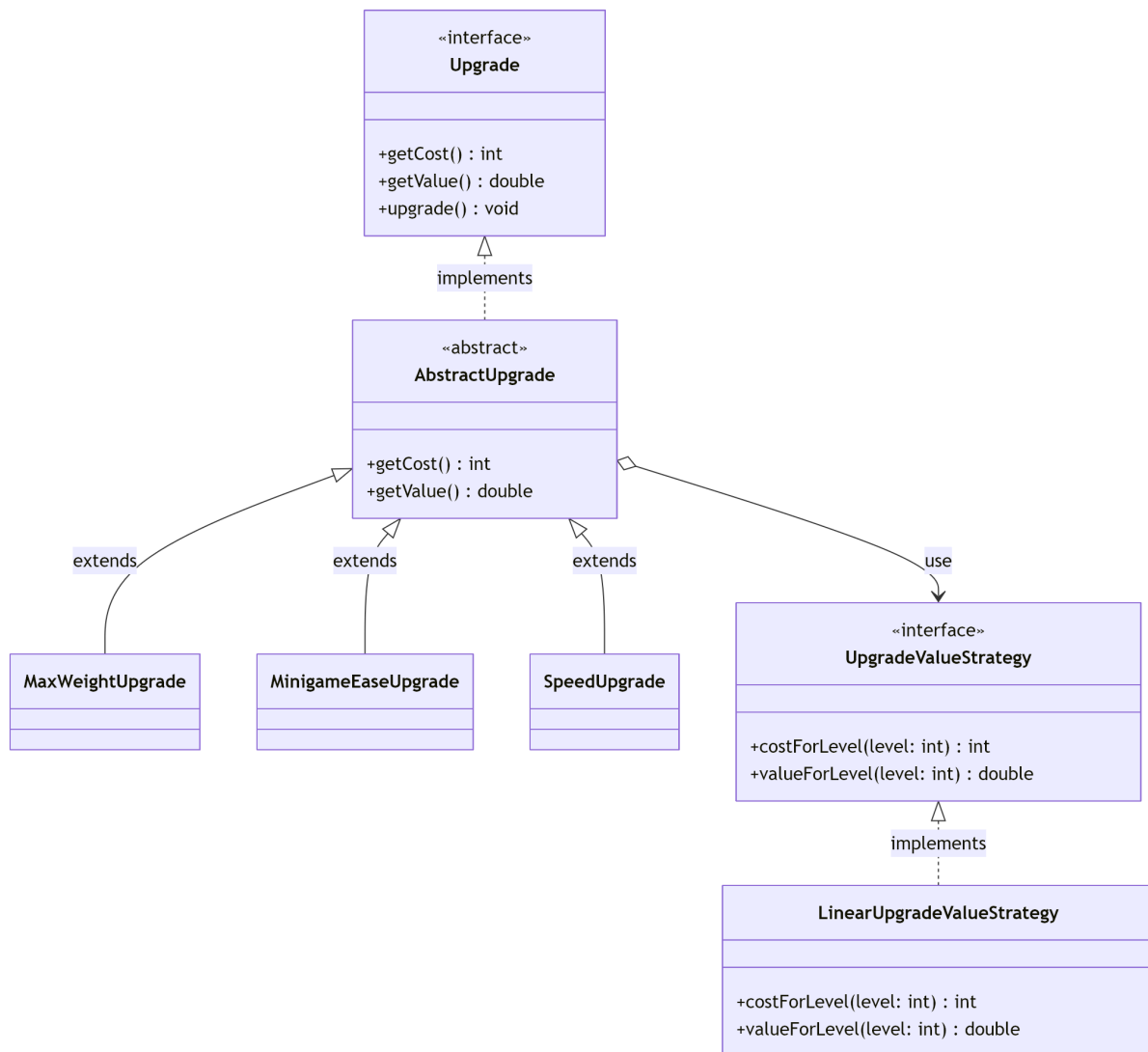
Pattern Observer: **Shop** è l'**Observable**, **UpgradeObserver** è l'interfaccia **Observer** implementata dai componenti che devono reagire a un potenziamento (es **Hook**). Quando viene comprato un nuovo potenziamento lo **Shop** invoca **notifyObservers()**, che propaga l'evento a tutti gli observer registrati.

Struttura dei potenziamenti

Problema: Ci sono diversi potenziamenti, e in futuro altri potrebbero essere aggiunti, è quindi necessario progettare una gerarchia che massimizzi il riuso del codice per evitare duplicazioni. Inoltre il costo di acquisto e il valore effettivo devono poter progredire in modi differenti per ogni potenziamento se necessario, anche qui senza implementare per ogni Upgrade il proprio sistema che porterebbe a molta duplicazione.

Soluzione: Ho deciso di definire un'interfaccia base **Upgrade**, seguita da una classe astratta che ne implementa i metodi comuni **AbstractUpgrade**, da cui derivano poi i potenziamenti concreti. Per risolvere invece il problema del calcolo di costo e valore ho deciso di

usare il pattern **Strategy** creando l'interfaccia **UpgradeValueStrategy** per delegare ad una strategia il calcolo.



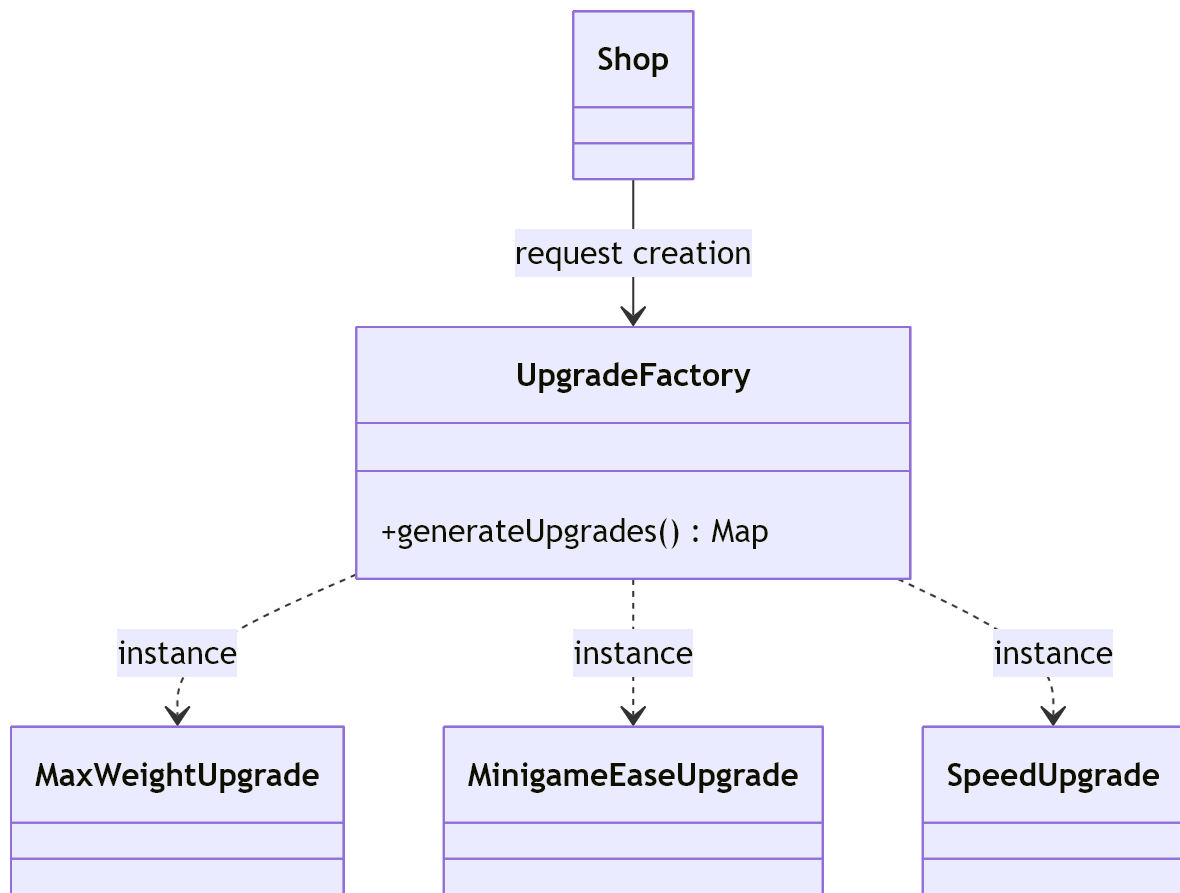
UML

Pattern Strategy: UpgradeValueStrategy è l'interfaccia che rappresenta la strategia, le sue implementazioni vengono poi usate da **AbstractUpgrade**.

Creazione dei potenziamenti

Problema: Essendoci diversi potenziamenti lo **Shop** non può istanziare direttamente le classi concrete senza causare un forte accoppiamento. Questo comporterebbe la necessità di effettuare modifiche ogni volta che si decide di aggiungere un nuovo potenziamento.

Soluzione: Adottare il pattern **Static Factory**. L'istanziazione degli oggetti viene delegata a una classe specializzata **UpgradeFactory**, in questo modo chi necessita dei potenziamenti non deve conoscere l'implementazione concreta o la classe specifica ma si limita a richiederli alla factory tramite apposito metodo.

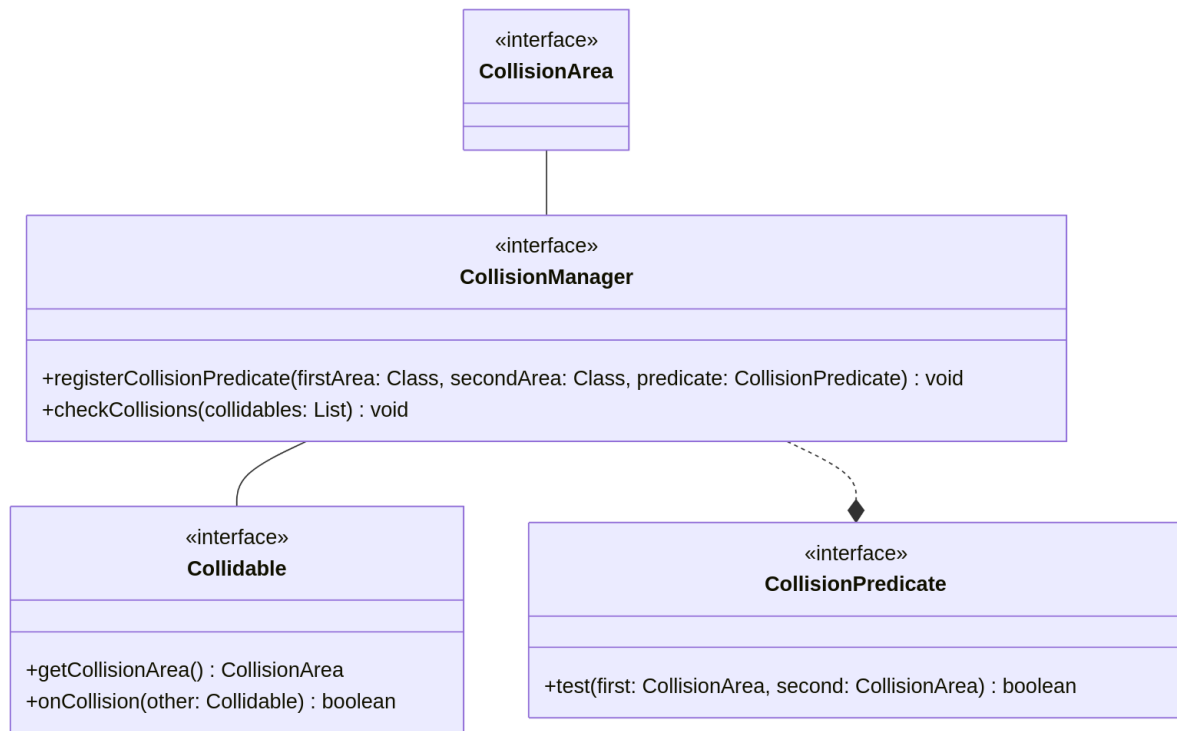


[UML](#)

Pattern Static Factory: `UpgradeFactory.generateUpgrades()` si occupa della creazione di tutte le istanze concrete di **Upgrade** e le restituisce a **Shop**.

Elia Soldati

Sistema delle collisioni



[Link](#)

Problema Il gioco prevede due tipi di collisioni: amo - pesce, e pesce - pesce. Date le differenti forme di queste entità ho ritenuto necessario ideare un sistema che fosse in grado di gestire hitbox di forme diverse.

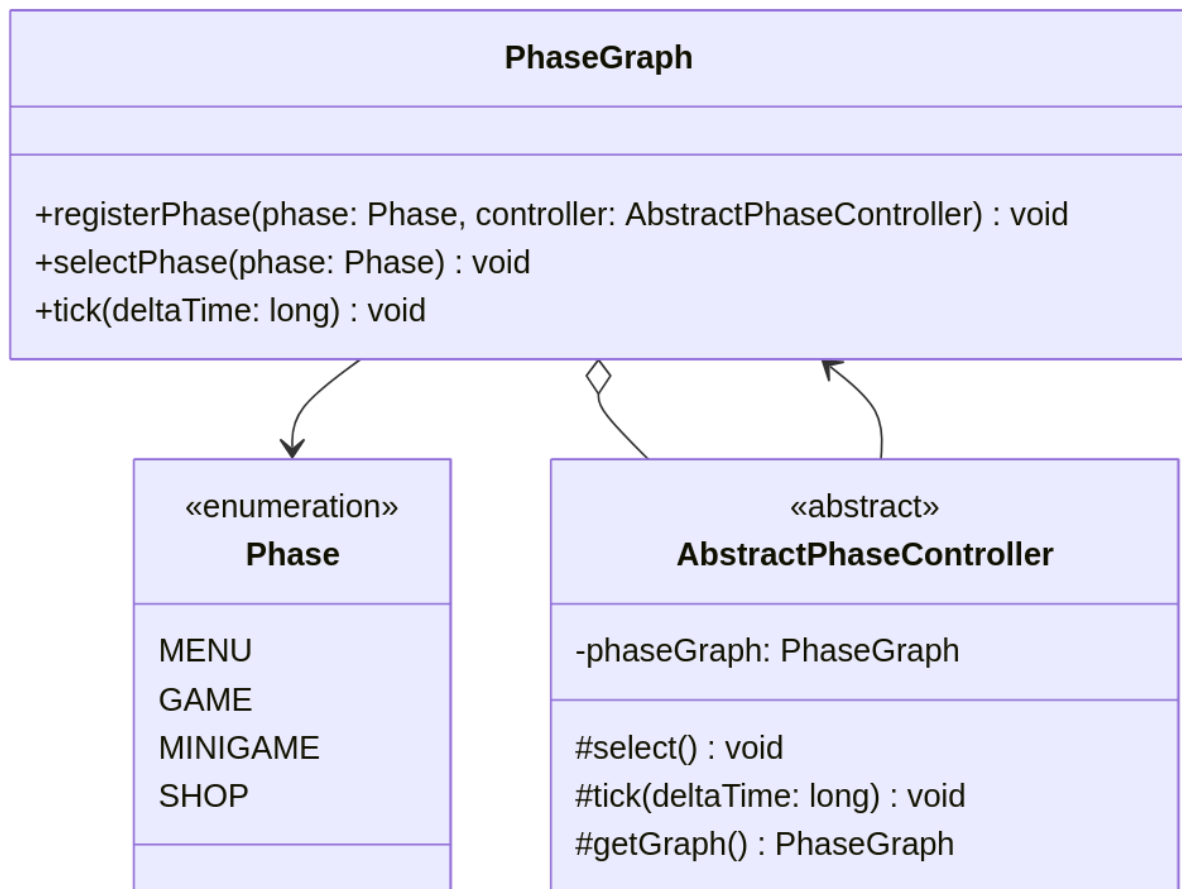
Soluzione Per realizzare il sistema ho scelto di mantenere un approccio abbastanza semplice: per permettere di disaccoppiarlo dal resto del model e renderlo quindi di fatto indipendente, esso opera su generici *Collidable*.

A ogni *Collidable* è richiesto di fornire la sua hitbox sotto forma di *CollisionArea*. Questo permette di mantenere l'intero sistema generico e funzionare con diversi tipi di hitbox, non solo con una specifica forma geometrica, come da requisito iniziale.

Ho scelto di seguire un modello stateless: il sistema delle collisioni non tiene traccia della posizione delle entità in tempo reale, ma bensì fornisce al suo utilizzatore un'interfaccia per determinare le collisioni istantanee partendo da un dato elenco di *Collidable*.

Il sistema controlla quadraticamente $O(N^2)$ ogni coppia di *Collidable* utilizzando specifici *CollisionPredicate*, ognuno dei quali è responsabile per una specifica coppia di *CollisionArea* (rettangolo - rettangolo, cerchio - cerchio, rettangolo - cerchio).

Controller phase system

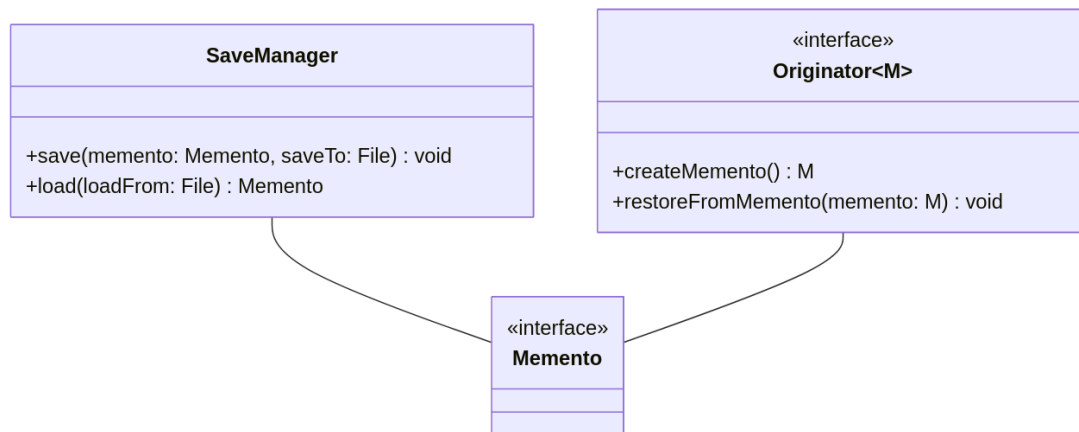


[Link](#)

Problema Il gioco si articola naturalmente in 4 fasi che si alternano: menu, gioco, minigioco e negozio. Al variare delle fasi, varia anche come l'input dell'utente viene gestito e anche come viene fatto avanzare lo stato del model (per esempio durante il menù lo stato del model non deve avanzare in quanto stato di pausa). Gestire tutto questo in un'unica classe diventerebbe caotico.

Soluzione Per risolvere il problema ho ritenuto appropriato lo sviluppo di un sistema che astragga maggiormente il susseguirsi di queste fasi, di modo che durante lo sviluppo di una fase ci si debba preoccupare solo di essa e non anche delle altre. Ho scelto di basare questo sistema sullo **State Pattern**, che è perfetto per rappresentare questo flusso in maniera simile ad una macchina a stati. In questo caso il *PhaseGraph* rappresenta la classe Context del pattern, mentre *AbstractPhaseController* rappresenta lo State. Questa struttura permette ad ogni fase di poter semplicemente richiedere un cambio di fase al *PhaseGraph* senza doversi preoccupare di come questo avvenga effettivamente sia da un punto di vista del controller che della view.

Sistema di salvataggio



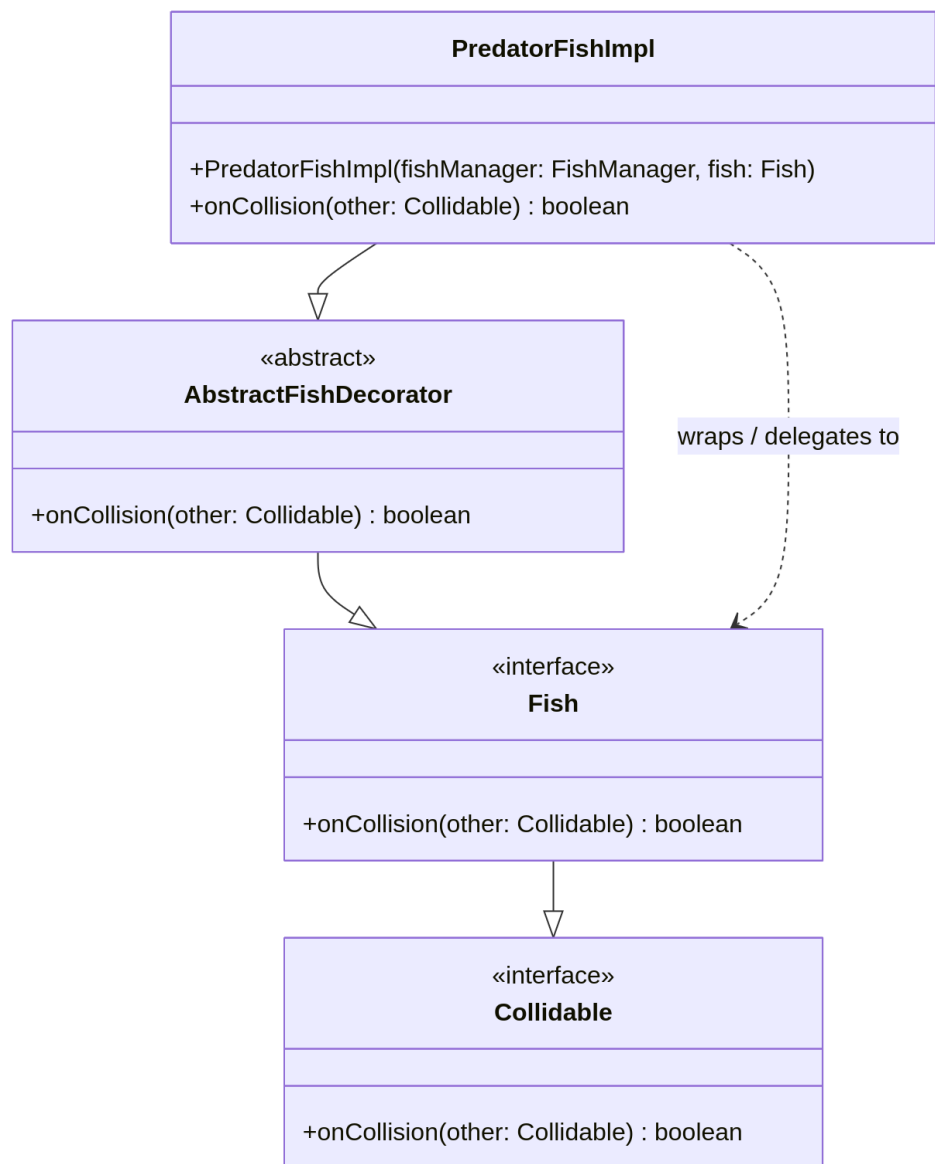
[Link](#)

Problema Si vuole salvare lo stato del gioco, tuttavia serializzare direttamente le classi usate dal model, o ancora peggio direttamente l'intero model è una pessima idea in quanto queste potrebbero contenere dati volatili aggiuntivi che è meglio non salvare, soprattutto se si salvano solo alcune parti del model, in quanto questi dati potrebbero fare riferimento ad altre parti del model che non salvate.

Soluzione Per risolvere questo problema torna estremamente utile il **Memento Pattern** che permette di creare delle "istantanee" di un oggetto. Abbiamo quindi gli *Originator*, ovvero un qualunque oggetto di cui possiamo creare (e ripristinare!) un'istantanea e i *Memento*, ovvero le istantanee stesse. Ogni *Originator* definisce il suo *Memento* e quindi cosa salvare e cosa ripristinare, in questo modo si lascia ad ogni specifico componente la scelta su quali dati è giusto salvare. Il cosiddetto Caretaker del pattern è rappresentato in questo caso dal controller che è il responsabile, all'input dell'utente, di richiedere al model il memento da salvare e poi utilizzare il *SaveManager* per effettuare l'effettiva scrittura su disco (speculare per il caricamento di un salvataggio!).

Ho scelto di utilizzare il protocollo standard di serializzazione degli oggetti di java. Non è probabilmente il più indicato per il salvataggio di un gioco, ma la sua semplicità e comodità penso che lo rendano un ottimo candidato per un progetto di queste dimensioni.

Pesci predatori



[Link](#)

Problema I pesci predatori dovevano trovare spazio all'interno di un sistema di gestione dei pesci e delle collisioni già esistente.

Soluzione Siccome un pesce predatore non è altro che un normale pesce ma con un comportamento aggiuntivo nelle collisioni, ho deciso di utilizzare il **Decorator Pattern**, perfetto per questo utilizzo in quanto permette di aggiungere comportamento ad un normale pesce, ma delegando all'implementazione originale il resto del lavoro, evitando di dover reimplementare tutte le funzionalità.

Quando un pesce collide con un altro pesce il decorator intercetta per primo questa collisione, eventualmente "mangiando" l'altro pesce (se quest'ultimo è di peso inferiore) e poi delega al pesce che si sta "decorando".

Sviluppo

3.1 Testing automatizzato

FishImplTest: verifica il comportamento dell'entità Fish, coerenza tipo/nome, peso individuale entro l'intervallo atteso, calcolo di valore economico e difficoltà di cattura come funzione del peso, effetto del moltiplicatore di velocità, validità dell'area di collisione, e gestione dei casi limite.

FishManagerTest: verifica il ciclo di vita della popolazione di pesci, immutabilità della vista restituita da **getFishes()**, assenza di effetti collaterali nel rimuovere un pesce non gestito, corretta transizione da vivo a morto e propagazione degli effetti di un cambiamento meteo sulla velocità dei pesci.

FishSpawnerTest: Vengono verificate posizione sempre esterna ai bordi orizzontali, coerenza tra lato di ingresso e direzione assegnata, rispetto della fascia verticale consentita in base all'altezza reale dell'hitbox, corretto filtraggio delle specie esclusive da temporale; e corretto wrapping dei predatori in **PredatorFishImpl**.

PositionTest: verifica il comportamento del value object **Position**, incluse coordinate negative e nulle.

MovementTimeTest: verifica che il fattore di scala del movimento sia coerente con il tempo trascorso tra un aggiornamento e l'altro a garanzia dell'indipendenza del movimento dei pesci dal frame rate effettivo del ciclo di gioco.

HookImplTest: copre tutti gli stati dell'amo, il movimento libero su entrambi gli assi(incluso il movimento diagonale e il blocco se due direzioni opposte sono attive), i limiti di posizione, la gestione delle collisioni con i pesci, la gestione del peso massimo di cattura, la reazione degli eventi upgrade e di cambiamento di meteo, l'emissione degli eventi verso i **FishingListener** registrati.

FishingMinigameImplTest: verifica la logica del minigioco a tempismo, il successo o il fallimento in base alla posizione dell'indicatore rispetto alla zona di bersaglio, il congelamento dell'esito dopo il primo tentativo, la relazione tra la difficoltà di cattura del pesce e l'ampiezza della zona bersaglio.

WeatherSystemImplTest: verifica la validazione dei parametri del costruttore, lo stato iniziale, il fatto che non si ripeta mai 2 volte lo stesso meteo di seguito, il recupero di più cambiamenti quando il tempo trascorso fra un aggiornamento e l'altro è molto grande, la notifica corretta a più osservatori contemporaneamente e che dopo che un osservatore è stato rimosso non gli arrivino più notifiche.

PlayerWalletTest: verifica che le transazioni del giocatore avvengano correttamente.

ShopTest: verifica che un potenziamento possa essere comprato solo se non si è raggiunto il massimo livello e se si hanno abbastanza soldi.

UpgradeFactoryTest: verifica la corretta istanziatura dei potenziamenti.

UpgradeTest: verifica ogni potenziamento, e che i potenziamenti non possano raggiungere un livello più alto del massimo livello impostato.

LinearUpgradeValueStrategyTest: verifica che i valori di costo e valore del potenziamento calcolati per il livello siano corretti.

CollisionManagerTest: testa il funzionamento del sistema delle collisioni. In particolare vengono verificati tutti i tipi di collisione implementati (rettangolo - rettangolo, cerchio - cerchio, rettangolo - cerchio). Viene poi testato che il sistema delle collisioni ignori effettivamente le entità che vengono rimosse durante la risoluzione delle collisioni. Infine viene verificato che il sistema sia resistente a modifiche della lista in input durante l'esecuzione.

PredatorFishImplTest: si testa il corretto funzionamento dei pesci predatori. In particolare si testa che solo il pesce più leggero sparisca, che il pesce non interagisca con altri Collidable che non siano pesci. Infine si verifica che il decorator deleghi effettivamente al pesce interno dopo aver eseguito la propria logica.

3.2 Note di sviluppo

Mattia Evangelisti

Non presenti.

Fabio Malucelli

Di seguito sono elencate le funzionalità avanzate del linguaggio Java e dell'ecosistema utilizzate per l'implementazione delle componenti di mia competenza (movimento dell'amo, sistema meteo, minigioco di cattura).

Uso di Stream

Verifica se un tipo di evento specifico è stato ricevuto da un observer di test tramite `Stream.anyMatch`, senza scrivere un ciclo esplicito con variabile di controllo booleana.

- `RecordingFishingListener`: [Permalink](#)

Uso di Lambda Expressions

Espressione lambda passata a Stream.anyMatch e espressioni lambda passate come Executable a JUnit assertThrows per verificare la validazione degli argomenti del costruttore di WeatherSystemImpl.

- RecordingFishingListener: [Permalink](#)
- WeatherSystemImplTest: [Permalink](#)
- WeatherSystemImplTest: [Permalink](#)

Matteo Raganini

Progettazione con generici:

- [View.java#L9](#)

Uso di lambda expressions:

- [BoidsManager.java#L71](#)
- [Shop.java#L46](#)
- [Shop.java#L57](#)

Uso di stream:

- [BoidsManager.java#L70](#)

Libreria JavaFX (indico i costruttori delle view):

- [GameView.java#L74](#)
- [MenuView.java#L56](#)
- [MinigameView.java#L57](#)
- [ShopView.java#L71](#)

Codice esterno:

- L'algoritmo Boids per i movimenti dei pesci è stato implementato seguendo quanto descritto [qui](#). [BoidsManager.java#L16](#) [Reynolds, Craig (1987). "Flocks, herds and schools: A distributed behavioral model". Proceedings of the 14th annual conference on Computer graphics and interactive techniques. Association for Computing Machinery. pp. 25–34.]

Elia Soldati

Per ogni funzionalità avanzata è riportato solo un permalink, anche se a volte potrebbe essere presente anche in ulteriori parti del codice.

Scrittura di metodi con parametri generici covarianti:

- [CollisionManager.java#L30](#)

Utilizzo di lambda functions:

- [GameWorldImpl.java#L46](#)

Commenti finali

4.1 Autovalutazione e lavori futuri

Mattia Evangelisti

Ruolo all'interno del gruppo: Il mio contributo al progetto si è concentrato sull'implementazione delle specie di pesci, del loro spawn e della gestione dei loro movimenti e hitbox. Mi ritengo complessivamente soddisfatto del lavoro svolto anche se mi sono reso conto che le prime volte che cominciavo a lavorarci sopra mi sentivo un po' perso, ma con il passare del tempo e con la concretizzazione del progetto ho iniziato a capire come muovermi, dunque realizzare piccole variazioni o nuove funzionalità non mi sembrava più così difficile.

Punti di forza: Mi ritengo soddisfatto in particolare per la scelta di applicare lo strategy pattern alla gestione dei movimenti permettendomi, nel corso dello sviluppo, di aggiungere nuovi comportamenti senza mai dover toccare altre classi.

Punti di debolezza: Il percorso di sviluppo non è stato però privo di difficoltà, alcune delle quali mi hanno causato svariate ore o giorni di lavoro. In particolare, l'integrazione con moduli sviluppati da altri membri del gruppo e altri errori dovuti al merge hanno richiesto svariati giorni di bugfix e riscrittura di classi intere, spesso dovuto dall'inesperienza di fare un lavoro vero e proprio in gruppo. Un'altra difficoltà significativa è emersa nella gestione dei confini della mappa e dell'andamento dei pesci che ha causato errori bizzarri come teletrasporti improvvisi e pesci che facevano il moonwalking.

Fabio Malucelli

Ruolo all'interno del gruppo: mi sono occupato delle dinamiche di pesca: il movimento dell'amo, il minigioco di cattura e l'introduzione delle variazioni climatiche. Il mio lavoro è partito da un design più articolato, con una barca e un amo separati e quattro stati (IDLE, DROPPING, REELING, MINIGAME) che modellava un lancio automatico con discesa e risalita, per poi evolvere verso un amo unico a movimento libero su entrambi gli assi, con stati ridotti a solo due valori. Ho inoltre progettato il sistema meteo, pensato fin dall'inizio per essere osservato indipendentemente sia dall'amo sia dal sottosistema di gestione dei pesci, così che i due potessero reagire alle stesse variazioni climatiche senza conoscersi a vicenda. Ho infine progettato il minigioco di pesca vero e proprio creando un Quick Time Event con un indicatore che si muove lungo una barra che determina la riuscita o meno della cattura di un pesce, e la sua difficoltà calcolata da vari fattori.

Punti di forza: ritengo che il punto di forza principale sia l'uso motivato dei design pattern. Ho applicato l'Observer in due punti concettualmente distinti (variazioni climatiche e eventi dell'amo), lo Strategy pattern per differenziare il comportamento dell'indicatore del minigioco in base al meteo e il Factory Method per centralizzare la creazione del minigioco. Ritengo

altrettanto un punto di forza l'attenzione costante alla separazione delle responsabilità tra model e controller, ogni volta che una decisione di dominio viene presa avviene nel model, così da ridurre il controller a pura delega degli input e degli esiti verso la view.

Punti di debolezza: riconosco di essermi concentrato quasi esclusivamente sul model, restando meno coinvolto nell'assemblaggio finale del controller e del game loop, delegando questa parte ad altri membri del gruppo. Ritengo inoltre che il design della parte di mia competenza sia maturato più per le iterazioni successive che per una pianificazione iniziale pienamente definita; diverse scelte strutturali (tra cui il modello di movimento e l'unità di misura del tempo tra i vari sottosistemi) sono state riviste e sostituite in corso d'opera invece di essere anticipate fin dalla prima progettazione. Questo mi ha permesso di allenarmi a modificare il codice in modo pulito senza introdurre regressioni, ma un'analisi più approfondita all'inizio avrebbe probabilmente ridotto il numero di modifiche strutturali necessarie.

Matteo Raganini

Ruolo all'interno del gruppo: Mi sono occupato del sistema dei potenziamenti e relativo shop, e della comunicazione di questi potenziamenti agli altri elementi del gioco che ne hanno bisogno. Ho inoltre implementato le varie view e movimenti di gruppo tramite algoritmo boids.

Punti di forza: ritengo di aver usato in modo corretto i pattern che ho implementato e di aver raggiunto un buon livello di astrazione tale da permettere l'aggiunta di nuovi potenziamenti in modo "facile".

Punti di debolezza: i boids sono stati implementati in modo non proprio ottimale e non interagiscono al meglio con il resto del sistema, questo è dovuto a una non buona progettazione iniziale del sistema. Le view poteva essere migliorata esteticamente e offrire più feedback visivi in risposta alle azioni dell'utente.

Elia Soldati

Ruolo all'interno del gruppo: Come già specificato mi sono occupato del sistema delle collisioni, dei pesci predatori, del sistema di salvataggio e del controller del gioco. Ho partecipato attivamente alla progettazione iniziale e generale del progetto e ritengo di aver avuto un ruolo importante da questo punto di vista, in particolare per quanto riguarda la progettazione della struttura che gestisce il flusso dei dati fra il model e la view, sicuramente perché ho gestito il controller che è il componente che possiede questo compito, ma anche per quanto riguarda l'effettivo interfacciarsi dei componenti MVC.

Punti di forza: Ritengo di aver realizzato un controller e un sistema di salvataggio entrambi abbastanza robusti, soprattutto grazie all'utilizzo di specifici pattern OOP. Ritengo di essere riuscito a mantenere un controller che è allo stesso tempo semplice, soprattutto da leggere e seguirne il flusso grazie alla separazione in fasi, ma anche estremamente flessibile ed

estensibile. Anche il sistema di salvataggio ritengo di essere riuscito a mantenerlo ben flessibile, soprattutto sulla granularità di scelta sui dati da memorizzare.

Punti di debolezza: I punti di debolezza più forti secondo me sono principalmente il sistema delle collisioni e il game loop. Il sistema delle collisioni è quello che mi lascia più triste perché ha questa registrazione dinamica che richiede di avere un'implementazione per ogni coppia di tipi di hitbox che non è il massimo, ma soprattutto mi sarebbe piaciuto poter rafforzare di più l'interfaccia dal punto di vista dei tipi, ma purtroppo, che io sappia, con il type system di java non è possibile raggiungere il livello di granularità che avrei desiderato (questo non vuol dire che non si potesse comunque migliorare rispetto allo stato attuale). Invece per il game loop utilizzo l'AnimationTimer di JavaFX in modo che il controller giri sullo stesso thread delle view, per il fatto che il model non è thread-safe e avere un singolo thread semplifica il lavoro (in ogni caso i due thread non avrebbero potuto eseguire contemporaneamente, ma lì si sarebbe dovuti sincronizzare per ottenere alla fine un'esecuzione analoga a quella attuale).

4.2 Commenti per i docenti

Elia Soldati

Desidero portare all'attenzione dei docenti la configurazione gradle che da regolamento si richiede di utilizzare. In particolare penso onestamente che la configurazione dei linter sia estremamente aggressiva, risultando esagerata. Capisco la necessità di standardizzare lo stile di questi progetti, ma alcuni check sono esagerati secondo me.

Faccio un esempio: il check "IllegalType" di checkstyle, che rifiuta la definizione di variabili basate sul tipo concreto invece che sulle interfacce generiche, come la definizione di una variabile di tipo HashMap invece che usare Map. È vero che nella maggior parte dei casi è giusto utilizzare l'interfaccia generica, ma ci sono dei casi in cui invece questa non rispetta i requisiti perché magari si ha necessità di dipendere dallo specifico comportamento dell'implementazione. Non dico che il check debba essere rimosso interamente, ma quantomeno dovrebbe essere possibile sopprimerlo qualora sia effettivamente necessario (io, molto onestamente, non sono riuscito a trovare un modo di farlo). Nel nostro progetto mi sono trovato a serializzare una mappa, ma l'interfaccia Map non implementa Serializable, mentre la specifica implementazione come per esempio HashMap sì, ma trovandomi impossibilitato a forzare l'utilizzo di HashMap come tipo sono stato costretto a "sperare" che l'utilizzatore passasse una mappa che implementa Serializable: decisamente un pessimo design, ma obbligato dal linter.

Questo è solo un esempio, ma in generale trovo il linter troppo aggressivo.

Appendice A - Guida Utente

All'apertura del gioco si viene presentati con un menù, che sarà del tutto analogo al menù di pausa. Dal di qui è possibile avviare (o riprendere, nel caso della pausa) la partita. Per aprire il menù di pausa durante il gioco è necessario premere il tasto ESC



Una volta avviata la partita sarà possibile muovere l'amo utilizzando WASD e per pescare un pesce questo deve collidere con l'amo, il che fa iniziare il minigioco (come da figura). Si tiene a precisare che esiste un limite massimo di peso di un pesce affinché l'amo possa pescarlo (aumentabile con i potenziamenti), quindi non tutte le collisioni con i pesci portano al minigioco.

